

An underwater scene featuring a large, pinkish-purple jellyfish in the foreground, surrounded by various coral reefs and smaller jellyfish. The background is a clear blue ocean with some fish visible in the distance.

Operating Systems & System Software

Session 2.1

Mouli Sankaran

Session 2A: Focus

- **A Sequential Program**
- **What is a Process?**
 - **CPU Burst and I/O Burst**
 - **Process Context**
 - **Context Switch by the OS**

A Sequential Program

- All programs are sequential in that they **execute a sequence of instructions in a pre-defined order.**

$x = x + 1$



LOAD	R1,	x
ADD	R1	
STORE	R1,	x

- Related computations are grouped into procedures.
- One sequential program is a single flow of instructions, there can be jump instructions, subroutine calls, etc. too.
- We say that there is a **single thread of execution or control.**

A Sequential Program with I/O Wait

- Here, the program is waiting for a Char (from serial port) to be received, to proceed further.
- The main program would loop here until SerFlag is set indicating the arrival of a char, by the Serial ISR.

Loop in the main program

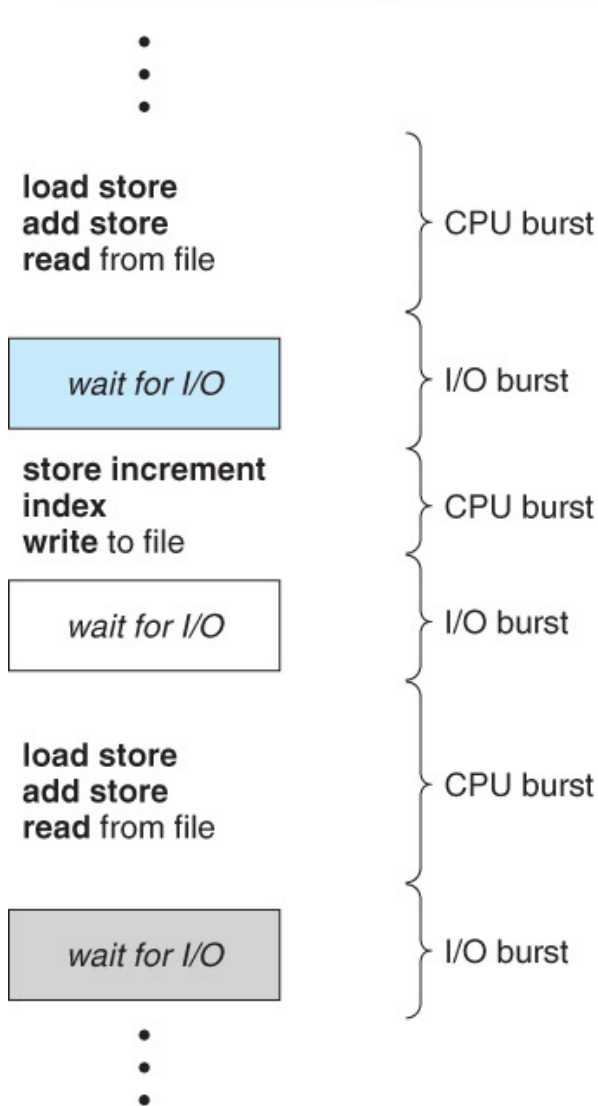
```
Loop: Load R7, SerFlag  
      CMP R7, 0  
      JMPEQ Loop  
      _____  
      _____
```

Note: R7 is loaded with the value of a **SerFlag variable** which is set to One when a Char is received on the serial port inside the Serial ISR

This main loop has to be looping till the char is received on the Serial port

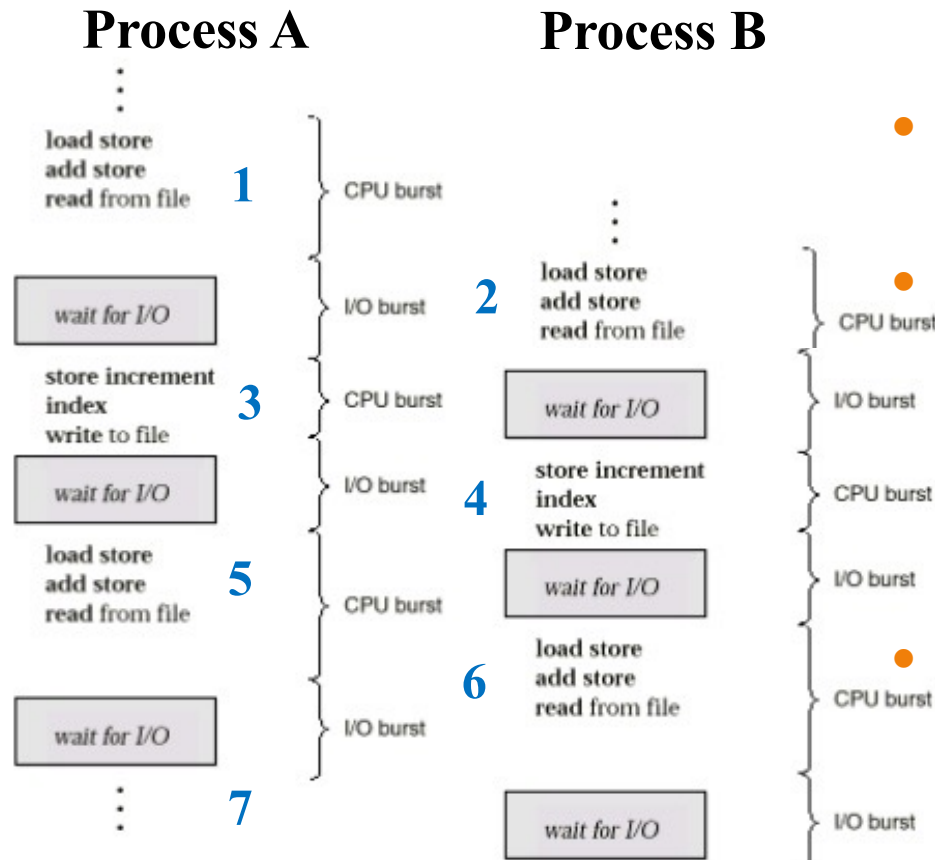
- With a single CPU, if this program is running, there is no useful job done by the CPU until a Char is received.
- This wastes the precious CPU cycles and power.

Program with File I/O Operations



- Here, the program operates on the data read from a file and updates the file.
- Then it waits for the file I/O write to be completed.
- Then again starts on another set of data by reading from the file.
- For every read and write operation with a file, a long wait is inevitable.
- As you are aware, file I/O is nothing but reading a file stored in a HDD and copying the contents to RAM for the program to operate on it.
- File read/write operations takes much longer than the speed of CPU execution.
- **CPU burst** is when the process is being executed in the CPU. **I/O burst** is when the CPU is waiting for I/O for further execution.

Using the time wasted by the CPU



CPU executes Processes A and B in this order: **A, B, A, B, A, B, etc.**
1, 2, 3, 4, 5, 6, 7, etc.

- Most of the programs in real-life have mix of CPU and I/O bursts.
- It will be good to use the time spent by the CPU waiting for I/O, effectively.
- If some way the state of the program which is waiting for the I/O can be saved somewhere, the CPU can be freed up to execute the “CPU burst” of another different program.
- With a single CPU, whenever the Process A is waiting for I/O it's status is saved into memory and
- Process B is brought into CPU to execute the “CPU burst” of Process B.
- Let us **understand** what is a **process ...**

Note: During I/O burst of a process, it is taken out of CPU, but the program resides in memory.

What is a Process?

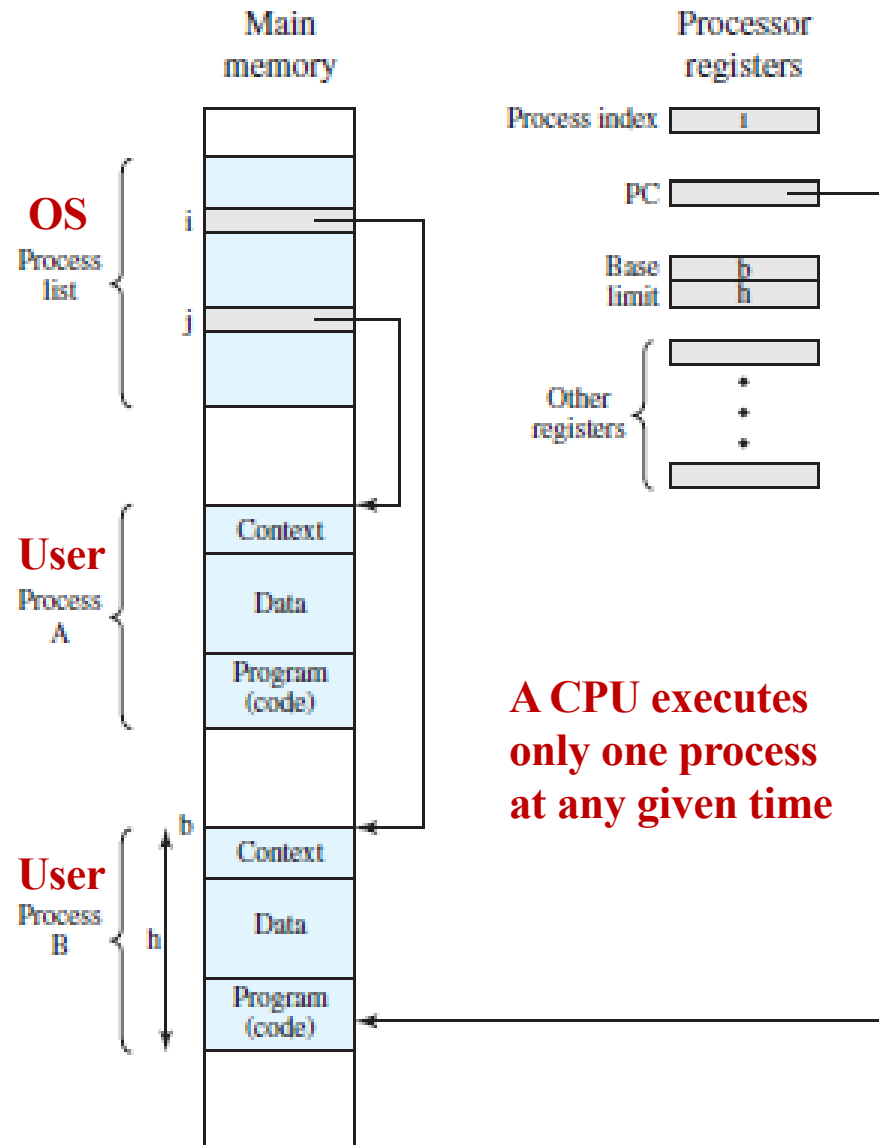
- The concept of process is fundamental to the structure of operating systems.
- The **most important task** of any modern operating system is **process management**.
- Process is a unit of executing software that is managed independently by the OS.
- Many **definitions** have been given for the term *process*, including
 1. A program in execution.
 2. An instance of a program running on a computer.
 3. The entity that can be assigned to and executed on a processor.
 4. A unit of activity characterized by a single sequential thread of execution, a current state, and an associated set of system resources.
 - Resources mean, for example, the files that a program uses. When a user opens multiple files using PowerPoint, from the OS perspective, PowerPoint process is using those files while it is executing.

Sample Implementation of Process in an OS

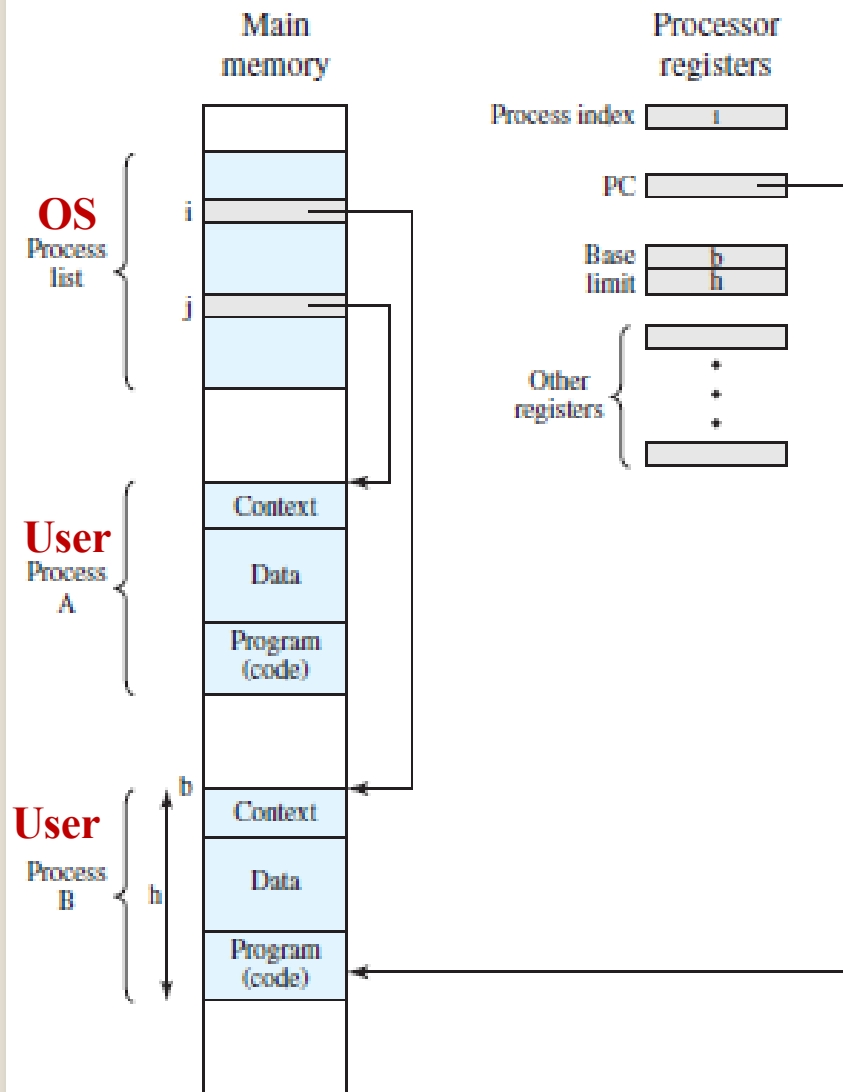
Reference: Ref1 book, page 83/820

Section 2.3 Major Achievements:

- OS allocates some memory for the resident sets of each process A and B.
- OS maintains an index into the start of the memory block allocated to each process.
- Each process is also given an unique ID by the OS, called Process ID.
- Assume, the CPU has two registers Base and Limit, which are loaded with the lower and higher addresses of the currently running process on the CPU.
- Process Index register of CPU has the index of the currently running process.
- Now, we need to understand “**context**” shown inside each process.

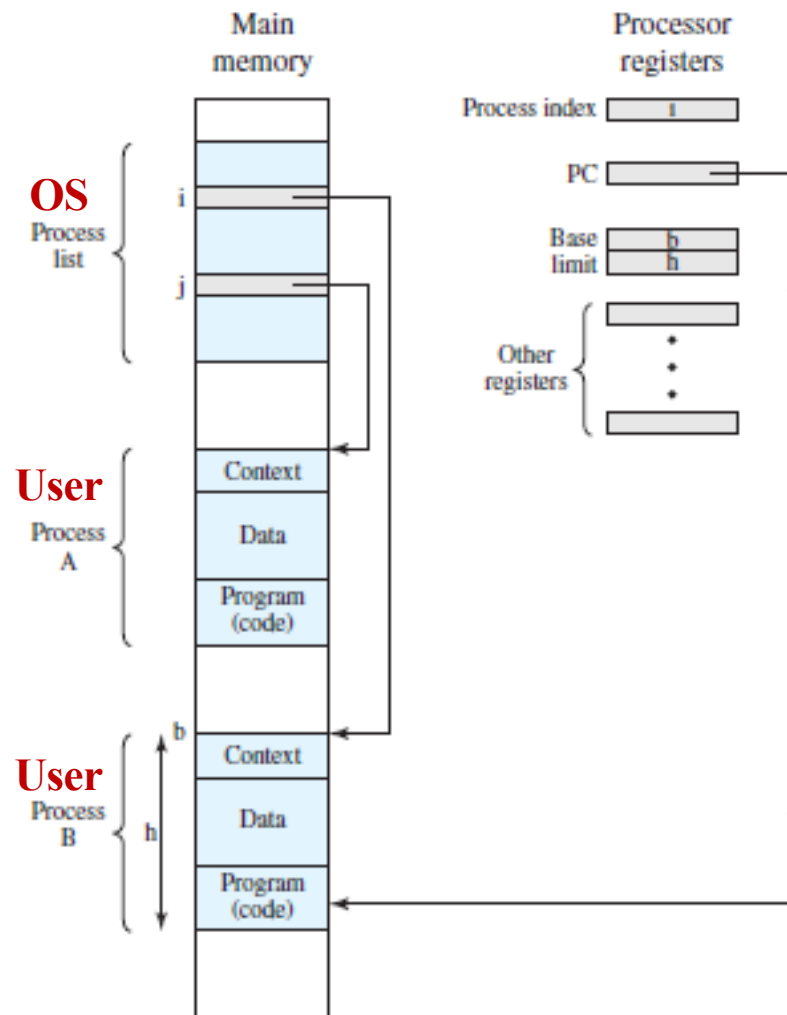


Process Implementation ... contd.



- What is a **Context** of a process?
- Imagine, the OS needs to remove a process from the CPU and bring in another process into CPU, to be executed.
- **What needs to be stored by the OS**, the status information of the process (**context**), which is exiting the CPU?
- OS needs to store the needed information of a process before removing it from the CPU.
 - Such that, when the OS brings back that process into the CPU again later, the process starts executing from the same place where it left off.
- One important thing to take care here is that correctness of the process execution is not compromised.

Process Implementation ... contd. a



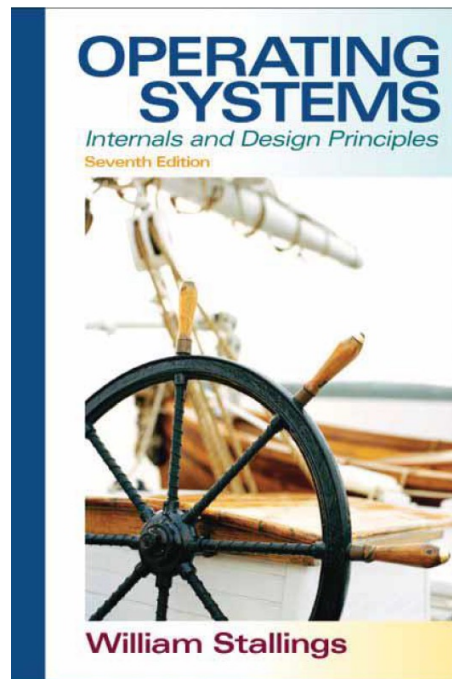
- What is a **Context** of a process?
- The **context** includes all the **contents** of the **registers** in the **CPU** when the process was forced to exit from the CPU.
- If the register values are all preserved, then the process which is taken out of CPU can be made to run again, by copying all those register values into the registers and allowing the process to run from the next instruction which is to be executed by the process.
- This entire sequence of operations done by the OS is called “**Context Switch**”.
- This is the most important activity done by the OS which is very closely tied to the CPU and always written as **assembly program**.

Session 2.1: Summary

- **A Sequential Program**
- **What is a Process?**
 - **CPU Burst and I/O Burst**
 - **Process Context**
 - **Context Switch by the OS**

References (1 to 5)

Ref 1



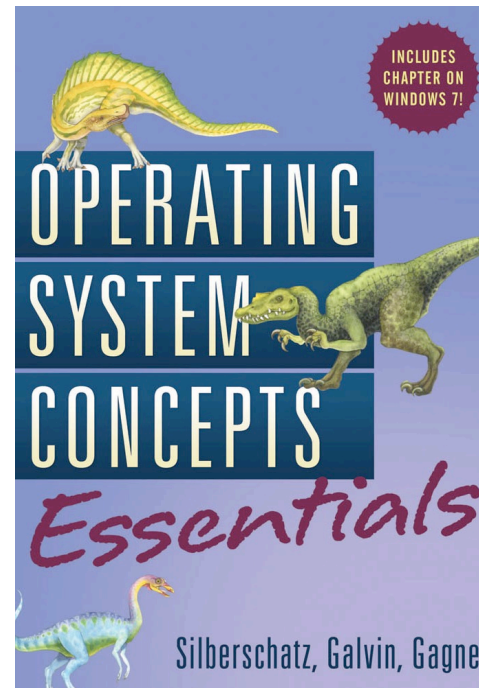
Ref 4

RP2040 A microcontroller by Raspberry Pi

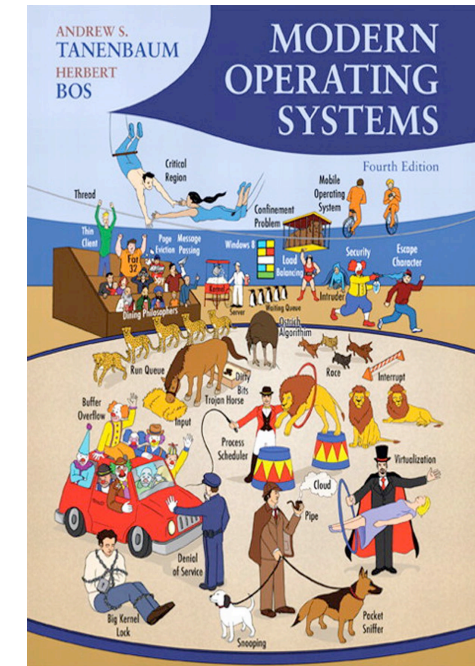
RP2040 Datasheet

A microcontroller
by Raspberry Pi

Ref 2



Ref 3



Ref 5

RP2040 A microcontroller by Raspberry Pi

Raspberry Pi Pico C/C++ SDK

Libraries and tools for
C/C++ development on
RP2040 microcontrollers